OhReally.nl

Leven als Rob in Frankrijk.

FREEBSD JAILS: A COMPLETE EXAMPLE

2021-02-08 Nerd stuff

FreeBSD jails are a great way to separate and compartmentalize processes, which enhances the security of your system. A jail is an enhanced chroot: it prevents an attacker who manages to compromise a service from gaining access to the rest of the system.

This post documents the setup of 2 jails that serve data to the outside world, and communicate between each other (through a *Unix Domain Socket*, not a *TCP socket*).

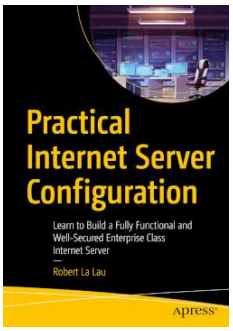
The 2 jails will serve websites and database respectively. Since many websites depend on a database, it could have made sense to install the web server and the DBMS in the same jail. But databases can also be used by other applications, and running multiple DBMS's would complicate the database replication and backup setup, so I decided to separate the web and database servers. This way, other applications don't need access to the web server jail to get to the database, and there is only a single database that needs to be backed up.

If you scroll down, it may look like a lot of work, but most of that only needs to be done once. When this is all done, it is rather simple to add new jails.

On the other hand, if you're looking for an instant solution that you can just copy and paste, then please move on and find another tutorial. This tutorial will make you reflect on your desired setup, and expect you to make some decisions for yourself.
If, like me, you are a person for whom the journey is at least as important as the destination, then this tutorial might be for you; if you just want to get things done as quickly as possible, you may want to try your luck elsewhere.

One last note before we begin:
This tutorial documents a setup with a web server jail that is reachable from the internet, and a database jail that is not reachable from the internet. This is an interesting

MY FIRST BOOK!



Practical Internet Server Configuration

Order my first book now!

MON PREMIER LIVRE !

experiment, but in a production environment you could ask yourself if it is worth the extra work of maintaining a jail for the database, if the database is not exposed to the network…

Table of Contents

- Conventions
  - Prompt and commands
  - Network interface
  - Firewall
  - DNS
- Preparation
  - Firewall
  - Kernel
  - Exisiting services
  - Download base
  - Create directories
  - Connectivity
  - Enable jail service
- Implementation
  - The www jail
    - Directories and base system
    - Configuration on the host
    - Configuration in the jail
    - Back to the host
  - The db jail
    - Directories and base system
    - Configuration on the host
    - Configuration in the jail
    - Back to the host
    - Replication
    - Connect the www jail
- Maintenance
- Changelog

Conventions

Some conventions used in this tutorial.

Prompt and commands

In this tutorial, I will prepend the prompt with *host* if a command must be executed on the host system:

```
host# ls -l
```

Commands that must be executed in the jail are prepended with *jail*:

```
jail# ls -l
```

All commands are executed as root, unless explicitly specified otherwise.

Network interface

Installation et configuration d'un serveur internet

Commandez maintenant mon premier livre !

MIJN EERSTE BOEK!

Configuratie van een internetserver

Bestel mijn eerste boek nu!

RECENT POSTS

My external network interface is called *igb0*. If the name of your network interface is different, you should not forget to correct that throughout this document.

If you don't know the name of your network interface, type

```
host# ifconfig
```

Firewall

In this tutorial, the *pf* firewall is used; if your server uses another firewall, it is up to you to translate the setup described here to your machine.

Rules for other firewalls, as well as any other additions, corrections and remarks, are welcomed at tuto.jails@ohreally.nl.

DNS

This tutorial does not go into the configuration of the DNS. However, there is no difference between the DNS configuration for the setup described here, with jails, of for a more traditional setup, without jails.

Preparation

This tutorial expects you to feel comfortable with FreeBSD administration.

Firewall

If you don't have a firewall, go fix that first. This tutorial expects you to have one, and what's more, I'm not going to help you set up a server without a firewall; you should learn to walk before you try to run.

Kernel

This setup needs a kernel module that is not loaded by default.

```
host# kldload nullfs
```

To make sure the module is automatically loaded the next time the system is (re)booted, add this line to the file */boot/loader.conf*:

```
nullfs_load="YES"
```

Alternatively, you could rebuild the kernel, with this module built-in. But rebuilding the kernel may be a subject for another tutorial.

Exisiting services

FreeBSD: when did I last update my ports/packages?
9GAG: Hide anonymous posts
Olijfbrood
Pain perdu
Brood (ontbijt)

CATEGORIES

Aan de kook (105)
Hobo food (12)
Algemeen (14)
Houtsnippers (17)
Kunst (2)
Nerd stuff (37)
Random stuff (9)
Schietsport (1)
Verhuizing (9)

ELPEREGRINO.NL

Crowdfunding
Fijne feestdagen!
SPW'er in opleiding
De skyline van Conquereuil
Terug naar school
De vakantie zit erop
Paprikaatje en tomaatje
Nieuwe foto's uit de tuin
Kapot maar voldaan
Ui

Make sure existing services are bound to a specific IP address, and not to a wilcard address. This prevents services on the host system from responding if a jail is not responsive.

The *bind* or *listen* address can usually be set through the configuration files for the services in question. Since I don't know which services you have installed, and I can't (or don't want to) explain it for all services here, I will let you do this on your own.

To find out which services are listening on a wildcard address, execute the following command:

```
host# sockstat -l
USER     COMMAND    PID   FD PROTO  LOCAL ADDRESS         FOREIGN ADDR
…        …          …     …  …      …                     …
```

Check the *LOCAL ADDRESS* column. Addresses are of the form *IP_address:port*, and the wildcard character is the asterisk, so **:22* means that the service is bound to port 22 on a wildcard IP address. (Port 22 is *sshd*, which is confirmed by the value in the *COMMAND* column, so in this case you would change the *ListenAddress* variable in */etc/ssh/sshd_config*).

For more tips on this, see jail(8).

Make the services listen on your external, internet/network-facing interface.

Download base

The jails will share the kernel with the host system.
All other software that is needed for the functioning of the service running in the jail is installed inside the jail. This means that, apart from the service the jail is created for, the jail also needs a complete base FreeBSD system.

Go to ftp.freebsd.org, select your architecture, then the desired FreeBSD version, and download the file *base.txz*.

If you don't know the name of the architecture of your system, execute the following command:

```
host# uname -m
amd64
```

If the host system uses the standard *GENERIC* kernel, you can pick the FreeBSD version you like. If the kernel was customized, it depends on the *COMPAT_FREEBSD** options that were specified when the new kernel was configured.

```
host# uname -i
GENERIC
```

```
host# sysctl kern.conftxt | grep COMPAT_FREEBSD
options COMPAT_FREEBSDx
…
```

Obviously, I assume that the host system is up to date (as it should be).
Anyway, I think you'll run into problems if the system in the jail is newer than the host system.

Let's say, for the sake of this tutorial, that you saved the *base* tarball to */root/Downloads/base.txz*.

Create directories

Create a base directory where the jails will be installed.

The FreeBSD handbook suggests */usr/jail*, but the Filesystem Hierarchy Standard states that /usr is for read-only data. The data served from our jails (websites and database) is not read-only.
To me */srv/jail* makes more sense; however, the FHS states that /srv is for data and scripts, which means that you should probably not install (almost) complete operating systems there.
For this tutorial, we'll use */srv/jail* anyway; feel free to pick and/or create a directory to your liking, because the Filesystem Hierarchy Standard will probably not have an answer before 2050 (they're a bit lax in updating the FHS).

```
host# mkdir -p /srv/jail
host# chown root:wheel /srv/jail
host# chmod 750 /srv/jail
```

To keep the logs accessible in a centralized location, we'll create a directory on the host, where each jail will have a sub-directory for it's log files.

```
host# mkdir -p /var/log/jail
```

Connectivity

It is not necessary to obtain additional IP addresses to make the jails available to the outside world (and vice versa). Instead, we can create a virtual local network; we then NAT the addresses in this network to the outside, and redirect inbound external traffic to this network, over the host's external network interface. To do so, we clone the host's loopback interface, and assign it a number of IP addresses from a range reserved for private networks.

Add these lines to */etc/rc.conf*:

```
cloned_interfaces="lo1"
ifconfig_lo1_aliases="\
```

```
    inet 172.17.2.1/32 \
    inet 172.17.2.2/32 "
```

Then, restart the *netif* and *routing* services. Make sure you do this in a single command, or your server will become unreachable and must be rebooted.

```
host# service netif restart && service routing restart
```

IP forwarding must be enabled in the kernel:

```
host# sysctl net.inet.ip.forwarding=1
host# echo "net.inet.ip.forwarding=1" >> /etc/sysctl.conf
```

Now, to allow the jails to connect to the internet, the following lines must be added to the firewall config.
(I told you you should have configured your firewall first; without the firewall in place, you won't be able to connect your jails to the internet.)

```
if = "igb0"
extip = "198.51.100.156"
jailnet = "172.17.2.0/29"
nat pass on $if from $jailnet to any -> $extip
```

Remember: this is for *pf*; translate it if you use another firewall.
The first two lines should be in the top of */etc/pf.conf*, with the other macro definitions; the third line should be in the translations section, just before the packet filtering. The *extip* address should be replaced with the external IP address for your server; *igb0* should be replaced with the name for your external network interface.

I've enabled NAT for an entire /29, which gives us 14 IP addresses to play with (172.17.2.1-14). To use all of these, make sure to add them to the *ifconfig_lo1_aliases* variable above.

The above will enable the jails to download files from the internet. The connection from the internet to the jails will be done later, when the jails have been configured.

Enable jail service

Add the following line to */etc/rc.conf*:

```
jail_enable="YES"
```

Implementation

In the scenario described here, both jails will contain an entire base system. Another option, not described here, is the concept of so-called 'thin jails'. These thin jails share a single base system, which is installed in a separate 'base jail'. The advantage of this approach is that you only need to update a single base system to update the base system for all jails. A disadvantage, however, is that once you've updated this one base system,

you will have to run *mergemaster*/*etcupdate* and update all ports in all jails, to make sure things keep functioning as they should. I prefer to update each jail when I think it is time to update that particular jail, without being forced to also do the others; disk space is not expensive, and an extra *freebsd-update* for each jail is not that much work.

The *www* jail

We'll create the jail for the web server first.

Directories and base system

Create a directory for the new jail. We'll create a base directory with a sub-directory named *fs*; the jail system will be installed into the sub-directory, which leaves some room for additional files in the base directory.
Then, extract the downloaded base system.

```
host# mkdir -p /srv/jail/www/fs
host# tar xf /root/Downloads/base.txz -C /srv/jail/www/fs/
```

Also create the directory that will hold this jail's log files on the host.

```
host# mkdir /var/log/jail/www
```

If you prefer to install applications from the ports collection instead of packages (I do), some extra directories must be created for mounting the ports collection and storing the distfiles.

```
host# mkdir /srv/jail/www/fs/usr/ports
host# mkdir -p /srv/jail/www/fs/var/ports/{distfiles,packages}
```

Configuration on the host

The configuration for the jails is stored in the file */etc/jail.conf*; this file must be created if it doesn't exist, yet.

```
# Start with default values that will be used by all jails
# (unless overridden in a jail configuration).

# Commands to execute when the jail is started or stopped.
# These commands are executed inside the jail.
exec.start = "/bin/sh /etc/rc";
exec.stop = "/bin/sh /etc/rc.shutdown";

# Don't import any environment variables when connecting
# from the host system to the jail (except ${TERM}).
exec.clean = "true";
```

```
# Mount a devfs filesystem on /dev inside the jail.
mount.devfs = "true";

# Give each jail it's own SYSV IPC message, semaphore and shared memor
sysvmsg = "new";
sysvsem = "new";
sysvshm = "new";

# The jail is named www.
www {
    # Where to find the jail on the host system.
    path = "/srv/jail/www/fs";

    # The fstab containing the filesystems to mount before starting th
    mount.fstab = "/srv/jail/www/fstab";

    # The IP address (+ netmask) and hostname for the jail.
    # Since most (if not all) programming languages have commands to d
    # IP address and local hostname, these may be exposed to users of
    # The hostname is also used in the sender address for outgoing mai
    ip4.addr = "172.17.2.1/24";
    host.hostname = "www.jail.example.com";
}
```

The file */srv/jail/www/fstab* should contain the filesystems that are mounted on the jail before it is started; this file is in the same format as the */etc/fstab* file, see fstab(5).

```
/var/log/jail/www    /srv/jail/www/fs/var/log            nullfs    rw
```

If you install applications from the ports collection, you should also include these lines:

```
/usr/ports           /srv/jail/www/fs/usr/ports          nullfs    ro
/usr/ports/distfiles /srv/jail/www/fs/var/ports/distfiles nullfs    rw
/usr/ports/packages  /srv/jail/www/fs/var/ports/packages  nullfs    rw
```

The ports collection itself is mounted read-only, to avoid messing up things in the host system and in other jails. The distfiles are stored on the host system, to make them available to the host system and the other jails. The ports will be built in the jail's */tmp* directory.

Configuration in the jail

The file */etc/make.conf* on the host possibly already lists *make* options to set and unset when building software, so it may be a good idea to copy this file to the jail.

```
host# cp -p /etc/make.conf /srv/jail/www/fs/etc/
```

The file */etc/resolv.conf* should also be copied to the jail, so that it knows which DNS servers to use.

```
host# cp -p /etc/resolv.conf /srv/jail/www/fs/etc/
```

If *resolv.conf* contains a *nameserver* address *127.0.0.1*, you should delete that in the jail, or replace the IP address with the IP address of your external network interface.

And then it's finally time to create and boot up the jail.

```
host# jail -c www
```

This command is only used the first time the jail is started (*-c* means *create*). From now on, if you want to manually stop or start the jail, you should use these commands:

```
host# service jail stop www
host# service jail start www
```

Once the jail is created, it is started automatically (even though it doesn't immediately show up in tools like *ps* and *top*). To display a list of runing jails, type

```
host# jls
```

You can connect to the jail with the following command:

```
host# jexec www /bin/sh
```

You can then navigate and manage it like any other newly installed system. So you'll probably want to start by setting the root password (*passwd*) and the time zone (*tzsetup*), and running *freebsd-update*.
The *vi* and *ed* text editors are available. Before you install software from the ports collection, make sure to add these lines to */etc/make.conf* (in the jail):

```
WRKDIRPREFIX=/tmp
DISTDIR=/var/ports/distfiles
PACKAGES=/var/ports/packages
```

The mounts that were made through the file */srv/jail/www/fstab* do not show up as mounts in the jail; as far as the jail is concerned, they are just part of the file system. The log files that are stored in the jail's */var/log* directory, are also available in */var/log/jail/www* on the host system.

This jail's purpose is to host websites. This means that a web server and probably PHP should be installed.
For the websites themselves there are 2 possibilities:

1. Install the websites within the jail.

2. Install the websites on the host system, and mount the directory containing the websites on the jail, using *nullfs*.

Some things to consider when deciding where to install the websites:

- It is easier to backup/copy/move the entire jail if the websites are installed in the jail.
- Many instant web applications have auto-update features, or the possibility to upload images or other files; to utilise these, the web server and/or PHP-FPM need write privileges for at least part of the website directory. For this, the websites should be installed in the jail, or the website directory should be mounted read-write.
- If the websites are installed in the jail, or the website directory is mounted read-write, web developers no longer need access to the host system; accounts can be created in the jail, and an arbitrary port on the host system can be forwarded to the SSH port on the jail; accounts on the host system for the web developers can then be deleted, limiting shell access to the host system to sysadmins.
- If the websites are installed on the host system, they are more easily accessible from other jails (which may come in handy, for example, if you want to serve them as Tor hidden services from a Tor jail as well). However, you probably do not want to mount them read-write on multiple jails; mount them read-only, and edit them from the host system (or from a dedicated website-editing jail, where you mount them read-write).
- Mounting the website directory read-only makes it harder for an attacker to deface the website.

Several scenarios are conceivable where the websites themselves are served to the internet from a jail that only has read access to the files, while the admin interfaces are served from a different jail, with write access to the files, to a more select audience. This is left as an exercise to the reader.

If you decide to store the websites on the host system, you should stop the jail, add the mount to */srv/jail/www/fstab* on the host system, and then start the jail again.

To work around a problem with a read-only mounted file system and a web application with auto-update features, the file system could temporarily be remounted read-write to do the update, and then be remounted read-only again.

Remember that all the files within the jail can be edited from the host system, so if you decide not to create web developer accounts in the jail, you may not need to install software like *bash* and *vim*.

Attention:
If you do decide to create user accounts in the jail, make sure to explicitly specify their UIDs, and make sure that these do not overlap with UIDs on the host system!

> *If*
>
> - *a user named alice with UID 1005 exists on the host system, and*
> - *a user named bob with UID 1005 exists in the jail, and*
> - *user alice creates the file /srv/jail/www/blah.txt on the host*
> - *or*

> - *host directory /srv/something is nullfs mounted on the jail, and*
> - *user alice creates the file /srv/something/blah.txt on the host*
>
> *then this file will belong to user bob in the jail!*

When the web server and the websites have been installed and configured, and the web server is started (add the web server to */etc/rc.conf* as you would do on a regular server), the setup can be tested from the host system with a tool like *ncat*.

```
host# printf "HEAD / HTTP/1.1\r\nHost: www.example.com\r\n\r\n" | ncat
```

Or you could just request the default website with the help of *lynx*, *wget*, *curl*, *fetch*, etc.

```
host# lynx 172.17.2.1
```

Clearly, if the site depends on a database, it will show error messages, because the database jail has not yet been installed.

Back to the host

If the websites do not depend on a database, they can now be made accessible from the internet. If they do depend on a database, you may wish to wait a little.

Add the following rules to the firewall (again: this is *pf*; translate if necessary):

```
jailwww = "172.17.2.1"
wwwports = "{ 80 443 }"
rdr pass on $if proto tcp from any to $extip port $wwwports -> $jailww
pass quick from $jailwww to $jailwww
```

The first 2 lines define new macros, and should be in the top of the file.
The 3rd line redirects all inbound traffic on ports 80 and 443 to those same ports on the *www* jail; this line should be just above or below the *nat* line that was added before.
The 4th line allows traffic between the jail and itself.

When the firewall is restarted, the websites will be served from the jail, and not or no longer from the host system.

As far as the DNS is concerned, you can just have the website aliases (like *www.example.com*) point at the server's external IP address; the firewall will make sure all requests are forwarded to the jail.

Once the firewall and the DNS are in place, TLS certificates can be requested and installed as usual.

Stop and disable the web server on the host system.

```
host# service apache24 stop && service apache24 disable
```

Replace *apache24* with the name of your web server.

Since all web traffic is now forwarded to the jail, the web server could also be uninstalled from the host system.

If you created users in the jail, you could have the firewall forward an arbitrary port (e.g. 2201) on the host to port 22 on the jail. The users would then SSH to port 2201 on the host system's external IP address to connect to the jail. Their accounts on the host system could then be deleted or locked.

The *db* jail

The setup of the database jail is not very different from that of the web server jail.

If you plan to move an existing database from the host system to a jail, and you have replication set up, make sure to stop the replication before you begin (on both ends, if you have 2-way replication set up).

```
MariaDB> stop slave;
```

Stop and disable the DBMS on the host system.

```
host# service mysql-server stop && service mysql-server disable
```

Directories and base system

```
host# mkdir -p /srv/jail/db/fs
host# tar xf /root/Downloads/base.txz -C /srv/jail/db/fs/
host# mkdir /var/log/jail/db
host# mkdir /srv/jail/db/fs/usr/ports
host# mkdir -p /srv/jail/db/fs/var/ports/{distfiles,packages}
```

One additional directory is created on the host and mounted on the jail. This directory will contain the *Unix domain socket* that will allow the websites to communicate with the database, which invalidates the need for any network traffic to the *db* jail.

```
host# mkdir /srv/jail/db/sock
host# chown 88:88 /srv/jail/db/sock
host# mkdir -p /srv/jail/db/fs/var/run/mysql/sock
host# chown -R 88:88 /srv/jail/db/fs/var/run/mysql
```

It looks like *88* is the default UID for user *mysql*; we will verify later that this is the correct UID for the jail.

Configuration on the host

The general settings have already been defined in */etc/jail.conf*, so only the jail specific settings need to be added.

```
db {
    path          = "/srv/jail/db/fs";
    mount.fstab   = "/srv/jail/db/fstab";
    ip4.addr      = "172.17.2.2";
    host.hostname = "db.jail.example.com";
}
```

The *fstab* file resembles the one for the *www* jail, but includes the directory for the MariaDB socket. This directory should be writable, to allow the MariaDB server to create the socket on (re)start.

```
/var/log/jail/db     /srv/jail/db/fs/var/log             nullfs    rw
/usr/ports           /srv/jail/db/fs/usr/ports           nullfs    ro
/usr/ports/distfiles /srv/jail/db/fs/var/ports/distfiles nullfs    rw
/usr/ports/packages  /srv/jail/db/fs/var/ports/packages  nullfs    rw
/srv/jail/db/sock    /srv/jail/db/fs/var/run/mysql/sock  nullfs    rw
```

Configuration in the jail

To begin with, 2 files are again copied from the host.

```
host# cp -p /etc/make.conf /srv/jail/db/fs/etc/
host# cp -p /etc/resolv.conf /srv/jail/db/fs/etc/
```

(Remember to delete *127.0.0.1* from *resolv.conf*, if necessary.)

And then the jail can be created, and the root password and the timezone can be set, *freebsd-update* can be run, etc.

```
host# jail -c db
host# jexec db /bin/sh
```

Again, add the following lines to */etc/make.conf* (in the jail) before attempting to install software from the ports collection:

```
WRKDIRPREFIX=/tmp
DISTDIR=/var/ports/distfiles
PACKAGES=/var/ports/packages
```

Then, install the most recent MariaDB server. When that is done, verify that the UID and GID for the *mysql* user are the same as were set for the socket directory.

```
jail# grep '^mysql:' /etc/passwd | awk -F: '{print $3":"$4}'
```

If these are not the same as set before (*88:88*), stop the jail, correct the ownership for the */srv/jail/db/fs/var/run/mysql* directory and it's sub-directories, as well as for the socket directory, and restart the jail.

For this jail, like for the previous one, the decision needs to made whether the databases should be installed in the jail or on the host. Since the directory must be writable to allow MariaDB to create databases, indexes and log files, there is probably less reason to install the databases outside of the jail. If the directory for the databases is mounted from the host, it should be mounted read-write on */srv/jail/db/fs/var/db/mysql*.
If existing databases on the host system must be moved to the jail, do the following (on the host system):

1. Rename the directory */srv/jail/db/fs/var/db/mysql* to */srv/jail/db/fs/var/db/mysql.backup*.
2. Copy the directory */var/db/mysql* to */srv/jail/db/fs/var/db*; make sure to preserve the permissions.
3. Once everything has been verified to work (below):
    1. Delete the directory */srv/jail/db/fs/var/db/mysql.backup*.
    2. Empty the directory */var/db/mysql*.

Before MariaDB is started, 2 configuration variables must be changed in the jail.

In */usr/local/etc/mysql/my.cnf*:

```
[client-server]
socket = /var/run/mysql/sock/mysql.sock
```

This will make the MariaDB server install the socket in the directory that was mounted from the host, and that will be shared with the *www* jail. And it will tell the MariaDB client where to find the socket.

In */usr/local/etc/mysql/conf.d/server.cnf*:

```
[mysqld]
bind-address = 172.17.2.2
```

This will make the MariaDB server bind to the jail's IP address.

When these settings have been made, the MariaDB server can be started (add *mysql_enable="YES"* to */etc/rc.conf* in the jail, as you would do on a regular FreeBSD server).

You may wish to execute *mysql_secure_installation* before continuing.

Back to the host

For normal day to day use, no network traffic to the database is needed, so no firewall rules have to be made. If tools like *MySQL Workbench* are used, use the firewall rule for replication below as an example; make sure to only allow traffic from known and trusted addresses.

To have MariaDB/MySQL clients on the host system transparently connect to the MariaDB server in the jail, a setting must be modified in the file */usr/local/etc/mysql/conf.d/client.cnf*:

```
[client]
socket = /srv/jail/db/sock/mysql.sock
```

Do **not** set this in *server.cnf* or *my.cnf*, as this could make an accidentally started MariaDB server on the host overwrite the jail's socket, which would make a mess of things.

To avoid the MariaDB server on the host from being started accidentally, it could also be uninstalled. It may, however, be practical to keep the MariaDB client (with the configuration correction above).

Replication

If you had replication configured before on the host, this is how you pick up replication where you stopped it before:

1. If you had generated any TLS certificates on the host, you should copy these certificates to the same directory, but relative to the jail.
2. Configure the firewall to forward inbound traffic on port 3306 to the jail; this is only necessary if this server is a replication master.

   ```
   jaildb = "172.17.2.2"
   dbrepl = "203.0.113.45"
   rdr pass on $if proto tcp from $dbrepl to $extip port 3306 -> $jail
   ```

   Replace the *dbrepl* address with the actual IP address of the replica.

3. Restart the replication (on both ends, if you use 2-way replication).

   ```
   MariaDB> start slave;
   ```

Since nothing has changed from an external point of view, no changes need to be made on the replication partner.

Connect the *www* jail

And finally the *www* jail can be connected to the *db* jail.

Stop the jail, and create the directory where the directory containing the socket should be mounted.

```
host# service jail stop www
host# mkdir /srv/jail/www/fs/var/run/mysql
```

Since the web server only needs read privileges, the ownership of the directory does not need to be changed.

Then, add the mount to */srv/jail/www/fstab*; it should be read-only.

```
/srv/jail/db/sock   /srv/jail/www/fs/var/run/mysql   nullfs   ro   0
```

Set the path to the MariaDB socket in the PHP configuration file */srv/jail/www/fs/usr/local/etc/php.ini* (if *php.ini* does not exist, copy *php.ini-production* to *php.ini* first).

```
[Pdo_mysql]
pdo_mysql.default_socket = /var/run/mysql/mysql.sock


[MySQLi]
mysqli.default_socket = /var/run/mysql/mysql.sock
```

And restart the *www* jail.

```
host# service jail start www
```

It may be practical to install phpMyAdmin in the *www* jail.

Maintenance

The jails can be updated using tools like *freebsd-update*, *portsnap*, *portmaster* and *pkg update*. These tools can be executed on the host or in the jail:

```
jail# freebsd-update fetch
jail# freebsd-update install
```

```
host# freebsd-update -b /srv/jail/www/fs fetch
host# freebsd-update -b /srv/jail/www/fs install
```

Since the ports collection is mounted read-only on the jails, it should always be updated on the host system before the ports in the jail are updated.

If *fail2ban* or similar tools are used, their configuration may need to be updated to (additionally) track log files in */var/log/jail/** (on the host).

Depending on your mail server setup, you may need to modify the configuration for your SMTP server or add the jails' hostnames to your DNS server, if you want to be able to receive mail from the jails.

Changelog

- **2021-02-08**
  Initial publication

Republish

FreeBSD: which ports/packages did I update today? Warme chocolademelk

Theme by Think Up Themes Ltd. Powered by WordPress.